Journal of Computational Physics 285 (2015) 100–110

Contents lists available at ScienceDirect

Journal of Computational Physics

www.elsevier.com/locate/jcp

A more efficient anisotropic mesh adaptation for the computation of Lagrangian coherent structures

A. Fortin [a,*], T. Briffard [a], A. Garon [b]

[a] GIREF, Département de mathématiques et de statistique, Pavillon Vachon, 1045 Avenue de la médecine, Université Laval, Québec, G1V 0A6, Canada

[b] Département de génie mécanique, École Polytechnique de Montréal, C.P. 6089, Succ. Centre-Ville, Montréal, H3C 3A7, Canada

ARTICLE INFO

Article history:
Received 4 June 2014
Received in revised form 15 November 2014
Accepted 7 January 2015
Available online 16 January 2015

Keywords:
Lagrangian coherent structures
Error estimation
Adaptive remeshing
Anisotropic elements
Finite Time Lyapunov Exponent

ABSTRACT

The computation of Lagrangian coherent structures is more and more used in fluid mechanics to determine subtle fluid flow structures. We present in this paper a new adaptive method for the efficient computation of Finite Time Lyapunov Exponent (FTLE) from which the coherent Lagrangian structures can be obtained. This new adaptive method considerably reduces the computational burden without any loss of accuracy on the FTLE field.

© 2015 Elsevier Inc. All rights reserved.

1. Introduction

Lagrangian coherent structures (LCS) are extremely useful to put in evidence the complexity of a flow field. The Lyapunov exponents (see Ottino [17]) are largely used in dynamical systems but were only recently introduced in fluid mechanics by Haller [12] to reveal fundamental fluid flow structures such as vortices, separation and attachment surfaces in both laminar and turbulent flows. This exponent is defined as the logarithm of the largest eigenvalues of the Cauchy–Green stress tensor associated to the flow map. It characterizes the exponential divergence of nearby particles after a finite time thus the terminology Finite-Time Lyapunov Exponent (FTLE). The FTLE has to be computed at each node of potentially very fine meshes. It is therefore important to be as efficient as possible. The LCS are defined as local extrema of this FTLE field and are characterized by very sharp ridges. They are therefore very difficult to capture accurately on a computational grid.

In a recent paper, Garon et al. [15] proposed an anisotropic mesh adaptation method in order to compute the FTLE field as efficiently as possible. They introduced a system of ODEs for the computation of the displacement, its gradient and its Hessian (third order) tensor, interpolated continuously on the unstructured mesh and approximated by Lagrange polynomials at the element level. The size of the system is 12 for two dimensional applications and 30 in 3D. They showed that their approach is very accurate even on unstructured meshes, in opposition to classical methods using finite differences for the computation of the gradient (see Shadden et al. [18] for example). However, they have chosen linear Lagrange polynomials for simplicity, and also because anisotropic mesh adaptation on this type of elemental approximation has reached maturity.

* Corresponding author. Tel.: 1 418 656 3489; fax: 1 418 656 2817.
E-mail address: afortin@giref.ulaval.ca (A. Fortin).

http://dx.doi.org/10.1016/j.jcp.2015.01.010
0021-9991/© 2015 Elsevier Inc. All rights reserved.

We propose in this communication to slightly modify an adaptive method developed in Bois et al. [4,5] which does not necessitate the computation of the Hessian tensor. This reduces the size of the systems of ODEs to 6 in 2D and to 12 in 3D. This is a huge gain in computational time with respect to the method proposed in [15]. We show that very anisotropic meshes can be obtained with a similar accuracy. We also show that the same method can be used for quadratic discretizations of the flow map (and in theory to polynomials of any degree), here again in opposition to adaptation methods based on Hessian error estimates.

2. Computation of Lagrangian coherent structures

The computation of Lagrangian coherent structures starts with the definition of a flow map of the domain or of a part of the domain. In the examples presented in this paper, the flow map will be defined on the complete computational domain Ω. This requires to compute, for each node $\mathbf{X}$ of Ω, the trajectories of the associated particle:

$$\frac{d\mathbf{x}}{dt}(t) = \mathbf{v}(\mathbf{x}, t), \quad \mathbf{x}(t_0) = \mathbf{X} \tag{1}$$

over a finite time T. In the above equation, $\mathbf{x}(t)$ is the position at time t of the particle initially located at $\mathbf{X}$ at time t_0 sometimes denoted $\mathbf{x}_X(t)$ or simply $\mathbf{x}(\mathbf{X}, t)$. The vector $\mathbf{v}(\mathbf{x}, t)$ is the Eulerian velocity field. The Lagrangian displacement field is then defined as:

$$\mathbf{u}(\mathbf{X}, t) = \mathbf{x}(t) - \mathbf{X}$$

which is defined in the entire computational domain.

The FTLE field at finite time T is then related to the largest eigenvalue λ_{max} of the Cauchy–Green tensor $\mathbf{C}(\mathbf{X}, T) = \mathbf{F}^T \cdot \mathbf{F}$ where $\mathbf{F}(\mathbf{X}, T)$ is the non symmetric deformation gradient tensor defined by:

$$F_{ij} = \frac{\partial x_i}{\partial X_j} = \delta_{ij} + \frac{\partial u_i}{\partial X_j} \quad \text{or} \quad \mathbf{F} = \mathbf{I} + \mathbf{G}$$

where $\mathbf{G}$ is the displacement gradient tensor (sometimes denoted $\nabla_X \mathbf{u}$). More precisely, the FTLE is defined as:

$$\sigma(\mathbf{X}, T) = \frac{1}{2|T|} \log(\lambda_{max})$$

and it is not difficult to show that if $d\mathbf{X}$ is an infinitesimal vector issued from $\mathbf{X}$ at time $t = t_0$, it will evolve with time following the equation:

$$\|d\mathbf{x}\| = e^{\sigma(\mathbf{X},T)|T|} \|d\mathbf{X}\|$$

showing an exponential divergence of nearby trajectories when σ is positive.

Computing this largest eigenvalue at each node $\mathbf{X}$ provides a field of FTLE in Ω and it can be done in several ways. The simplest method consists in using finite differences to compute the components of the tensor $\mathbf{F}$ (see Shadden et al. [18] for example). This is a very reliable method on structured (cartesian) grids but it proved more or less accurate on unstructured meshes.

Garon et al. [15] proposed another approach more appropriate for unstructured grids. They modified system (1) to solve the following coupled system:

$$\begin{cases} \dfrac{d\mathbf{x}}{dt}(t) = \mathbf{v}(\mathbf{x}, t), \quad \mathbf{x}(t_0) = \mathbf{X} \\ \dfrac{d\mathbf{F}}{dt}(t) = \nabla \mathbf{v} \cdot \mathbf{F}, \quad \mathbf{F}(t_0) = \mathbf{I} \end{cases} \tag{2}$$

hence providing a direct computation of tensor $\mathbf{F}$. The above system of ODEs contains all the necessary information for the computation of the FTLEs on any type of mesh.

3. Mesh adaptation

To improve the accuracy of the solutions, error estimators and mesh adaptation can now be used. This should allow to substancially reduce the number of nodes while preserving, and most probably increasing, the accuracy of the numerical solution, especially if anisotropic meshes are considered.

3.1. Hessian based mesh adaptation

One of the most popular anisotropic adaptive methods is based on the definition of a solution dependent metric which is directly linked to the interpolation error. This metric can be built using different approaches and we refer to [6,9,21] for an overview of the possibilities. Many of these adaptation methods require the recovery of the Hessian matrix of the variable on which we want to adapt and the reader is referred to [8,13,14,11] for more details.

Estimating the Hessian matrix is not a trivial task. The most common approach consists in first estimating the gradient using methods such as those proposed by Zhu–Zienkiewicz [23,24] and Zhang–Naga [22], from which we can recover the Hessian.

The construction of an anisotropic mesh is usually achieved by modifying how the distance between two neighboring points is measured. This is realized by the construction, using the recovered Hessian of the solution, of a metric field, such that the distance between the vertices of a simplex (triangle, tetrahedron) is unity. Specifically, this methodology defines an optimal metric that yields a finite element mesh, with a target number of nodes (or equivalently the number of Lagrangian particles), minimizing the interpolation error.

In the specific case of the computation of an FTLE field, the first possibility is to adapt on the FTLE field itself ($\sigma(\mathbf{X}, T)$). We therefore need to estimate the gradient and the Hessian of $\sigma(\mathbf{X}, T)$ using one of the above recovery methods. This is a particularly delicate task since σ can present very steep variations as we shall see.

A better strategy is to adapt the mesh using the displacement $\mathbf{u}(\mathbf{X}, T)$ as proposed in [15]. This would normally require the recovery of the gradient and of the Hessian of the displacement field which is a third order tensor defined by

$$H^i_{kl} = \frac{\partial^2 u_i}{\partial X_k \partial X_l} = \frac{\partial^2 x_i}{\partial X_k \partial X_l}$$

having, taking into account the symmetries, 18 different components in 3D and 6 in 2D. It is however easily seen that the gradient of the displacement field ($\mathbf{G}$) can be directly computed by solving system (2) ($\mathbf{G} = \mathbf{F} - \mathbf{I}$). But we can go one step further and complete system (2) to include the Hessian:

$$\begin{cases} \dfrac{d\mathbf{x}}{dt}(t) = \mathbf{v}(\mathbf{x}, t), \quad \mathbf{x}(t_0) = \mathbf{X} \\ \dfrac{d\mathbf{F}}{dt}(t) = \nabla \mathbf{v} \cdot \mathbf{F}, \quad \mathbf{F}(t_0) = \mathbf{I} \\ \dfrac{dH^i_{jk}}{dt}(t) = \dfrac{\partial^2 v_i}{\partial x_m \partial x_l} F_{mk} F_{lj} + \dfrac{\partial v_i}{\partial x_l} H^l_{jk}, \quad H^i_{jk}(t_0) = 0 \quad \text{for } i, j, k = 1, \cdots, n \end{cases} \tag{3}$$

as also proposed in [15]. Once the above system has been solved for each particle $\mathbf{X}$, the Hessian of each component of the displacement is available from which different metrics (one per component) can be obtained. No gradient or Hessian recovery method is thus needed. These metrics are then intersected as described in Alauzet and Frey [1] to produce a single metric on which the mesh can be adapted. System (3) has a total of 30 ODEs in 3D and 12 in 2D. The results presented in [15] clearly show that this approach is very accurate and efficient when compared to classical methods.

3.2. Hierarchical adaptive method

We propose in this section a different adaptive method that does not require the computation of the Hessian matrices at all. This means that system (2) can be used (instead of (3)), thus reducing the size of the system of ODEs to 12 in 3D and 6 in 2D, a huge save in computational cost without any loss of accuracy.

We briefly recall the main ideas behind the hierarchical error estimator and we refer to [4,5] for more details. We denote $\mathcal{T}_h$ a triangulation of the domain Ω and K its elements and let V_h^k the space of continuous functions whose restriction to any element K of $\mathcal{T}_h$ belongs to the space $P^{(k)}(K)$ of polynomials of degree k. We therefore suppose that we have computed (using system (2)) an approximation of the displacement denoted $\mathbf{u}_h^{(k)}(\mathbf{x}) \in (V_h^k)^n$ (n being the space dimension), and an approximation of its gradient tensor denoted $\mathbf{G}_h^{(k)}(\mathbf{x}) \in (V_h^k)^n \times (V_h^k)^n$. To make things clear, for a linear finite element solution ($k = 1$), system (2) is solved at each vertex of the mesh while when $k = 2$ (quadratic finite element solution), system (2) is solved at each vertex and at each mid-edge node of the mesh and so forth.

Our objective now is to approximate the error $\|\mathbf{u} - \mathbf{u}_h^{(k)}\|$ where $\|\cdot\|$ is an appropriate norm (L^2-norm for instance). Hierarchical error estimators are based on the construction, starting from a given finite element solution $\mathbf{u}_h^{(k)}(\mathbf{x})$ of degree k, of a more accurate approximation $\hat{\mathbf{u}}_h^{(k+1)}(\mathbf{x})$ of degree $k+1$. The L^2-norm (or any other norm) of the error on an element K can then be approximated by the relation:

$$\left\|\mathbf{u} - \mathbf{u}_h^{(k)}\right\|_{0,K} \simeq \left\|\hat{\mathbf{u}}_h^{(k+1)} - \mathbf{u}_h^{(k)}\right\|_{0,K} \tag{4}$$

We now briefly describe the precise construction of $\hat{\mathbf{u}}_h^{(k+1)}(\mathbf{x})$. The idea is then to define on each element of the mesh:

$$\hat{\mathbf{u}}_h^{(k+1)}(\mathbf{x}) = \mathbf{u}_h^{(k)}(\mathbf{x}) + \mathbf{c}_h^{(k+1)}(\mathbf{x}) \tag{5}$$

where $\mathbf{c}_h^{(k+1)}(\mathbf{x})$ can be seen as a degree $k+1$ correction to the degree k approximate displacement $\mathbf{u}_h^{(k)}(\mathbf{x})$, thus the name hierarchical error estimator. The dimension of the correction space containing $\mathbf{c}_h^{(k+1)}$ is $\dim(P^{k+1}) - \dim(P^k) = k+2$ in the two-dimensional case and $(k+3)(k+2)/2$ in 3D. Hierarchical finite element basis, as described in Bank and Smith [2], can be used for that purpose. This is schematically illustrated in Fig. 1 for the cases $k = 1$ and $k = 2$ used in this work.

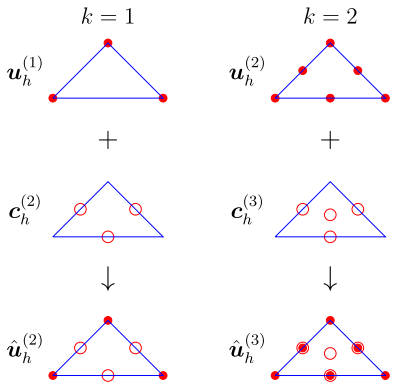


Fig. 1. $\mathbf{u}_h^{(k)} + \mathbf{c}_h^{(k+1)} = \hat{\mathbf{u}}_h^{(k+1)}$, for $k = 1, 2$.

A linear (resp. quadratic) solution is enriched by a quadratic (resp. cubic) correction to produce a new quadratic (resp. cubic) approximation.

In the following, we drop the subscript h to ease the notation. The construction of $\mathbf{c}^{(k+1)}(\mathbf{x})$ on each element is very simple and can be achieved component by component. To enrich $\mathbf{u}^{(k)}(\mathbf{x})$ in order to get an approximation $\hat{\mathbf{u}}^{(k+1)}(\mathbf{x})$ of degree $k+1$, we need, as in a Taylor expansion, its derivatives of order $k+1$. Obviously, the $(k+1)$th derivatives of $\mathbf{u}^{(k)}(\mathbf{x})$ vanish but we can use the kth derivatives of the gradient $\mathbf{G}^{(k)}(\mathbf{x})$. This means that the $(k+1)$th derivative of $\hat{\mathbf{u}}_h^{(k+1)}(\mathbf{x})$ should coincide with the appropriate kth derivative of $\mathbf{G}^{(k)}(\mathbf{x})$. Note that from Schwarz's theorem, the number of different $(k+1)$th derivatives is precisely $\dim(P^{k+1}) - \dim(P^k)$.

For example, in the linear case ($k=1$) the following system is obtained:

$$\begin{cases} \dfrac{\partial^2 \mathbf{c}_i{}^{(2)}}{\partial x_1^2} = \dfrac{\partial \mathbf{G}_{i1}^{(1)}}{\partial x_1}, \qquad \dfrac{\partial^2 \mathbf{c}_i{}^{(2)}}{\partial x_2^2} = \dfrac{\partial \mathbf{G}_{i2}^{(1)}}{\partial x_2}, \\ \dfrac{\partial^2 \mathbf{c}_i{}^{(2)}}{\partial x_1 \partial x_2} = \dfrac{1}{2}\left(\dfrac{\partial \mathbf{G}_{i2}^{(1)}}{\partial x_1} + \dfrac{\partial \mathbf{G}_{i1}^{(1)}}{\partial x_2} \right). \end{cases} \tag{6}$$

A similar system can be obtained for $k=2$ using third order derivatives of $\mathbf{c}_i{}^{(3)}$ and second order derivatives of $\mathbf{G}^{(2)}$. In the general case, a well posed linear system of dimension $\dim(P^{k+1}) - \dim(P^k)$ can be built on each element, whose solution completely defines $\mathbf{c}_i{}^{(k+1)}$.

Once this is done, from Eq. (4), the L^2-norm and H^1-seminorm of the error can be approximated on each element K as:

$$\left\| \mathbf{u} - \mathbf{u}^{(k)} \right\|_{0,K} \simeq \left\| \mathbf{c}^{(k+1)} \right\|_{0,K} \quad \text{and} \quad \left| \mathbf{u} - \mathbf{u}^{(k)} \right|_{1,K} \simeq \left\| \nabla \mathbf{c}^{(k+1)} \right\|_{0,K}$$

These quantities can be easily computed since $\mathbf{c}^{(k+1)}$ is a standard Lagrange finite element field. We now have an error estimation on each element and the global error estimate is then given by:

$$\left\| \mathbf{u} - \mathbf{u}^{(k)} \right\|_{0,\Omega} = \left(\sum_{K \in \mathcal{T}_h} \left\| \mathbf{c}^{(k+1)} \right\|_{0,K}^2 \right)^{1/2} \tag{7}$$

To obtain a new mesh, local operations such as node displacement, edge swapping, node elimination and edge refinement are used. The first objective is to reach a prescribed target level of error (e_Ω) in L^2-norm. More specifically, we modify the mesh, using edge refinement and node elimination, so that:

$$\sum_{K \in \mathcal{T}_h} \left\| \mathbf{c}^{(k+1)} \right\|_{0,K}^2 = e_\Omega^2$$

As a second objective, we also try, using node displacement and edge swapping, to reach some form of equidistribution of this error. Ideally, we would like the absolute value of the error to be essentially constant ($|\mathbf{u} - \mathbf{u}^{(k)}| = c$) throughout the domain. To attain this last objective, we minimize, as in D'Azevedo and Simpson [10], the L^2-norm of the gradient of the error (the H^1-seminorm of the error) approximated by the quantity:

$$\left(\sum_{K \in \mathcal{T}_h} \left\| \nabla \mathbf{c}^{(k+1)} \right\|_{0,K}^2 \right)^{1/2}$$

Ultimately, if the L^2-norm of the gradient of the error vanishes, this means that the error is constant. The minimization is performed using edge swapping and node displacement. Remarkably, the minimum on the H^1-seminorm of the error

cannot be achieved without somehow reorienting and stretching the elements in the appropriate direction, therefore leading to anisotropic meshes as shown in [10,4,5].

From the above discussion, this means that:

$$e_\Omega^2 = \int_\Omega \left|\mathbf{u} - \mathbf{u}^{(k)}\right|^2 dx = c^2 \text{ meas } (\Omega) \quad \text{and thus} \quad c^2 = \frac{e_\Omega^2}{\text{meas } (\Omega)}.$$

This implies that on each element K, the local error e_K should satisfy:

$$e_K^2 = \left\|\mathbf{c}^{(k+1)}\right\|_{0,K}^2 = \frac{e_\Omega^2 \text{ meas } (K)}{\text{meas } (\Omega)}$$

If this target value can be reached locally on each element, then the global target error e_Ω will be attained and the error will be equidistributed. This is done by sweeping the nodes (for node elimination and node displacement) and the edges (for edge division and edge swapping) a number of times until the two objectives are approximately satisfied. A new displacement field is then computed and the process is repeated until the mesh no longer changes.

This overall strategy is sufficient to produce strongly anisotropic meshes where the solution allows it. It also leads to optimal (minimal error per unit area or volume) in the sense described in [10]. From a practical point of view, for a given target error, we can expect a mesh with a minimal number N of computational nodes (CNs). For a linear discretization ($k = 1$), the computational nodes are the vertices of the mesh. For a quadratic discretization ($k = 2$), the computational nodes includes both the vertices and triangle (tetrahedron) mid-edge nodes. Obviously, the absolute optimal mesh is not always reached since we modify and optimize the mesh in a heuristic way but our experience shows that we are quite close.

4. Numerical results

We present in this section two numerical examples to illustrate our method. The first one is two-dimensional and will be presented with all details. We then present a three-dimensional example showing that the method works equally well in this case.

The efficiency of the procedure described in the previous sections is studied with an analytical flow field of class C^∞ in space and in time. The particles are allowed to move freely in $\mathbb{R}^n$ when the mesh is adapted to the time-evolving structures in the computational domain. Consequently, the flow field yields a well defined FTLE field (see [15]).

In the numerical experiments, the velocity field and its derivatives are computed from analytical functions to avoid the introduction of any bias introduced by interpolation and reconstruction of derivatives from a discrete approximation of the velocity field. This implies that the displacement field and its derivatives depend only on the accuracy of the time-stepper. For a better control of this error, we have used the fourth order ODE45 solver from Matlab (see Moler [16]). The time step was allowed to vary in order to guarantee an absolute value of the error smaller than 10^{-6}. Smaller error bounds (10^{-7}) were also tested but no significant differences were observed.

Since we are using a finite number of particles to define a mesh, the L^2-norm of the error behaves as $E \approx O((\Delta t)^4, N^{-k/n})$. The first term corresponds to the integration error over time, while the second, with $k = 2$, results from the linear finite element interpolation of the displacement field (respectively $k = 3$ with quadratic interpolation). If the time step is well chosen, the discretization error over time becomes negligible with regard to the finite element approximation, and we should expect $E \propto N^{-k/n}$, at each time step, when computing the interpolation error with the procedure described in Section 3.2.

Under these optimal conditions, we will specifically examine, in this section, the reliability of the mesh adaptation software of adapting to the FTLE field through our proposed error estimate of the flow map. Analytical examples will be used for this purpose.

4.1. Solomon and Gollub's analytical Rayleigh–Bénard model

In this example (see Solomon and Gollub [19]), the Eulerian velocity field is obtained from the stream function:

$$\psi(x_1, x_2, t) = \sin\big(\pi\left(x_1 - g(t)\right)\big)\sin(\pi x_2) \quad \text{and thus} \quad \mathbf{v} = \left(\frac{\partial \psi}{\partial x_2}, -\frac{\partial \psi}{\partial x_1}\right)$$

where $g(t) = 0.3\sin(4t) + 0.1\sin(2t)$. The computational domain is:

$$\Omega = \left\{(x_1, x_2) \in \mathcal{R}^2 \mid (0 \le x_1 \le 2)\ \&\ (0 \le x_2 \le 1)\right\}$$

The flow map is constructed starting at $t_0 = 0.5$ and the integration time is $T = 1$ (thus $t_{final} = 1.5$).

As a first approach, system (2) is solved at all computational nodes of the mesh using the ODE45 solver of order 4. The average time step was around 0.0038 ensuring an absolute value of the error smaller than 10^{-6}. Hence, this choice results in a displacement field $\mathbf{u}$ virtually independent of the temporal discretization. The adaptive strategy of Section 3.2 is then used to adapt the mesh using the estimated error on the displacement field $\mathbf{u}(\mathbf{X}, T)$.

Table 1
Rayleigh–Bénard model: estimated error on the displacement vector.

Linear discretization ($k=1$)			Quadratic discretization ($k=2$)		
Number of vertices	Number N of CNs	$\|\hat{\mathbf{u}}_h^{(2)} - \mathbf{u}_h^{(1)}\|_{0,\Omega}$	Number of vertices	Number N of CNs	$\|\hat{\mathbf{u}}_h^{(3)} - \mathbf{u}_h^{(2)}\|_{0,\Omega}$
19382	19382	5.4×10^{-3}	5464	21711	5.1×10^{-4}
37999	37999	2.3×10^{-3}	13049	51951	1.5×10^{-4}
114224	114224	7.1×10^{-4}	21078	84051	8.8×10^{-5}
220230	220230	3.6×10^{-4}	56445	225240	1.7×10^{-5}

Table 1 shows the L^2-norm of the estimated error (using (7) on the displacement for linear and quadratic discretizations. It also includes the number of computational nodes (N) for each case. The evolution of the error is better seen in the convergence curves of Fig. 2(a). As the mesh is refined, the estimated error decreases at a rate very close to the optimal one for both discretizations. In the linear case, the L^2-norm of the error should behave as $O(h^2)$ in the linear case and $O(h^3)$ in the quadratic case. Since the number N of computational nodes behaves as $h \sim 1/N^{1/n}$, n being the spatial dimension, this should translate as $O(N^{-1})$ and $O(N^{-3/2})$ convergence respectively. Our numerical experiment yields a convergence rate of -1.1 (linear) and -1.44 (quadratic), very close to the expected theoretical behaviour. As expected, the quadratic discretization provides more accurate solutions for a given number of computational nodes. For example, for our last linear mesh with $N = 220230$, we get an estimated error in L^2-norm around 3.6×10^{-4}. The last mesh having a similar number of computational nodes (225240) in the quadratic case yields an error around 1.7×10^{-5}, more than one order of magnitude smaller. This is a significant improvement and it demonstrates the usefulness of higher order error estimators.

As can be seen in Fig. 3, the computed FTLE fields are smooth even though the associated meshes are very anisotropic. In each case, four different meshes were computed but only the second ones are illustrated. The sharp variations of the FTLE field are well captured and only very small amplitude oscillations can be observed when looking very closely. These oscillations disappear on the subsequent finer meshes. For comparison purposes, we present in the same figure a quadratic solution on a very fine regular (uniform) mesh with 261057 CNs, a number much larger than those of our adapted meshes. It is easily seen that even the FTLE field computed with a linear discretization on an adapted mesh with only 37999 CNs is less oscillatory. The steep ridges characterizing the FTLE fields are much better captured when the mesh is adapted.

In a second approach, we adapt the mesh on the FTLE field instead of the displacement. This approach was explored previously by Bélanger [7]. The adaptive method is essentially the same but in Eq. (4), the displacement field is replaced by the scalar FTLE field. The most important difference is that we now need to estimate the first order derivatives of the FTLE field which no longer come from the solution of system (2). A close look at Fig. 3 indicates that the estimation of the derivatives of a function presenting such sharp variations will not be an easy task. Accurate gradient recovery techniques must then be used. This also means a slight increase in computational time. In this work, we will use the least square recovery method described in Zhang–Naga [22]. Note however that our adaptation method only requires first order derivatives which is much less sensitive than Hessian based adaptation which would require estimations of second order derivatives of the FTLE field as in [7].

The results are presented in Table 2 and Fig. 2(b) for the convergence curves. In this last figure, it can be clearly seen that the convergence curves are no longer optimal, though they are still acceptable. The computed slopes are -0.87 in the linear case and -1.2 in the quadratic case. We believe this is due to the estimated derivatives which are much less accurate when using recovery techniques. Note however that since we are mainly interested in the FTLE field, adapting directly on that field is certainly reasonable. Despite the fact that the mesh is now directly adapted on the FTLE field, it behaves globally as it did with the adaptation on the displacement field (see Fig. 4), though some small differences in the distribution of vertices can be seen locally. Close views of the meshes in the top left and top right corners of the computational domain are presented in Fig. 5. Both approaches are able to accurately capture the FTLE field as presented in Fig. 6.

4.2. Kelvin cat's eyes model

Kelvin cat's eyes (see Stuart [20]) models a periodic array of vortices. In this case, the velocity is obtained from the stream function:

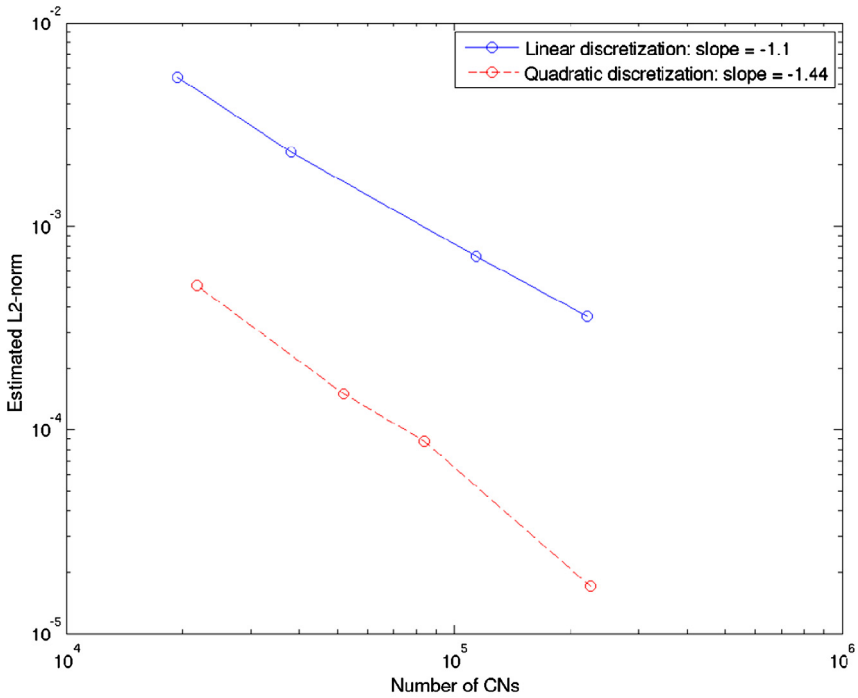
$$\psi(x_1, x_2, t) = -u_c\left(x_2 + \frac{\lambda}{2\pi}\ln\left[\cosh\left(\frac{2\pi}{\lambda}x_2\right) - a\cos\left(\frac{2\pi}{\lambda}(x - u_c t)\right)\right]\right)$$

where $a = 1/2 = u_c$ and $\lambda = 0.2$. The computational domain is:

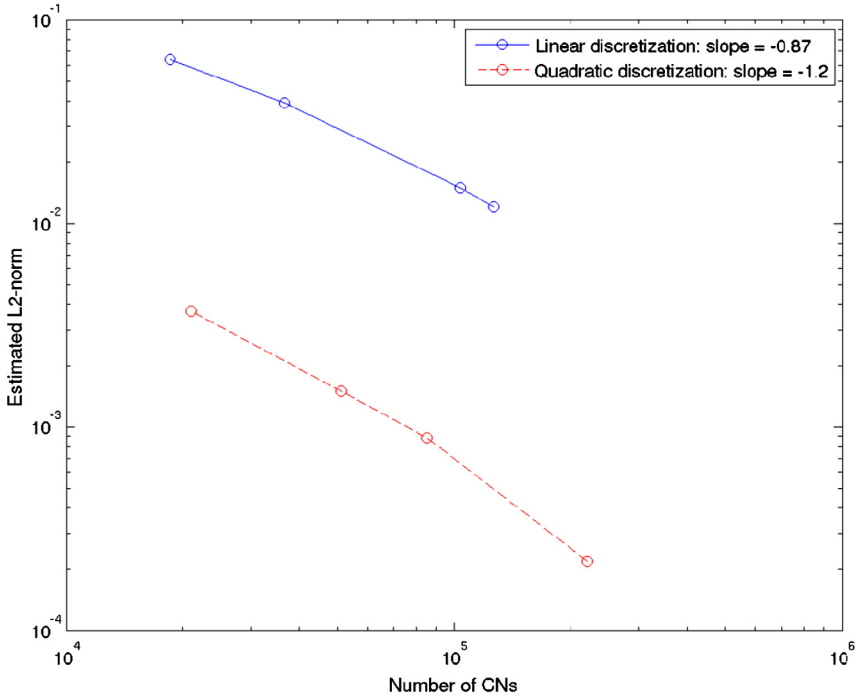
$$\Omega = \left\{(x_1, x_2, x_3) \in \mathcal{R}^3 \mid (0 \le x_1 \le 1)\ \&\ (-0.5 \le x_2 \le 0.5)\right\}$$

The flow map is constructed starting at $t_0 = 2$ and the integration time is $T = -0.6$ (thus $t_{final} = 1.4$).

For this example, we present the results in the quadratic case only since they are significantly more accurate. The meshes were adapted on the FTLE field but similar results were obtained (but not presented) while adapting on the displacement field. The solution and the final mesh are presented in Fig. 7. The row of vortices can be easily seen and the mesh closely



(a) Meshes adapted using the displacement field



(b) Meshes adapted using the FTLE field

Fig. 2. Convergence curves.

follows the intricate pattern of the FTLE field. This mesh was obtained after 4 iterations, that is 4 computations of the FTLE field and 4 remeshings. The two-dimensional mesh contains 4700 vertices and close to 19 000 CNs. It is therefore rather coarse but the solution is still very well captured.

Our adaptation method works equally well in the three-dimensional case. To illustrate this, we will solve once again the Kelvin cat's eyes problem but in three dimensions. The computational domain is now:

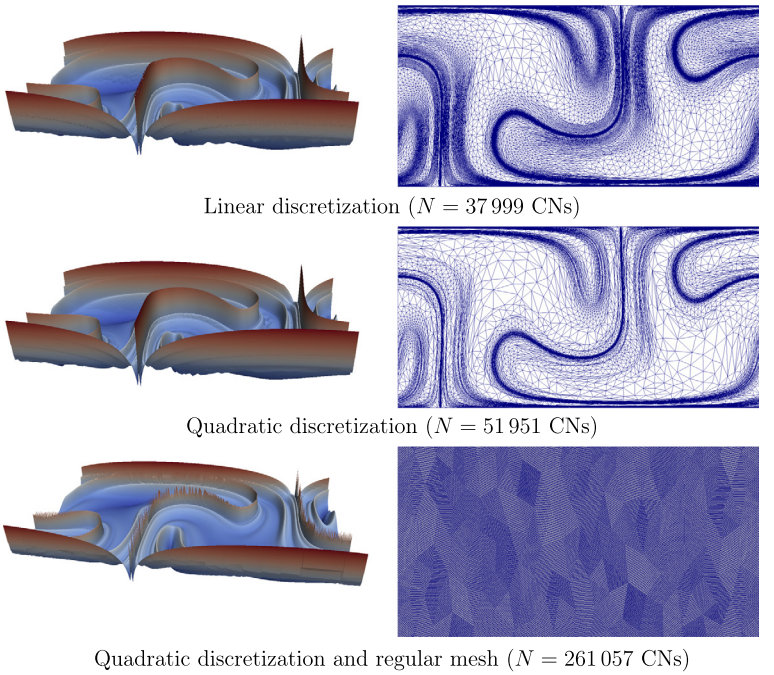


Fig. 3. Computed FTLE fields and associated meshes adapted using the displacement field.

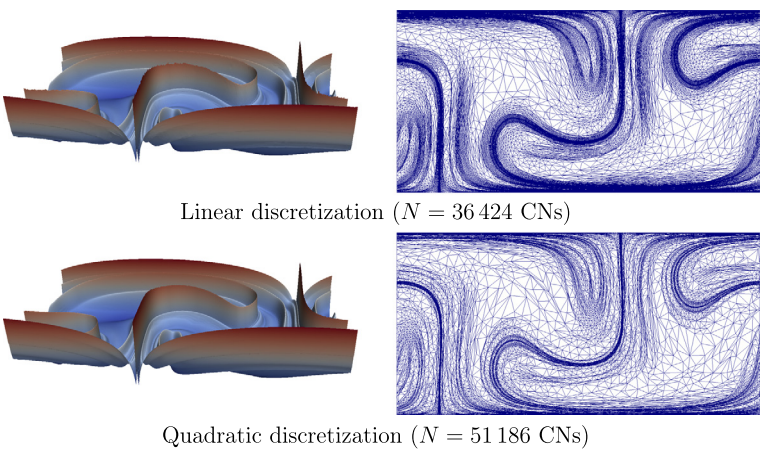


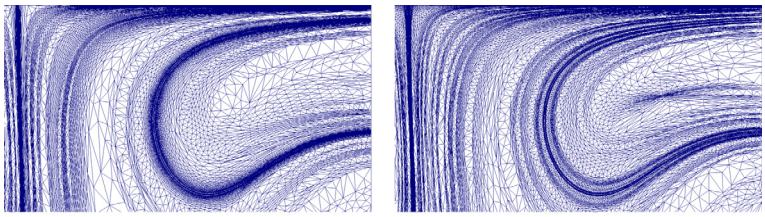
Fig. 4. Computed FTLE fields and associated meshes adapted using the FTLE field.

Table 2
Rayleigh–Bénard model: estimated error on the FTLE field.

Linear discretization ($k = 1$)			Quadratic discretization ($k = 2$)		
Number of vertices	Number N of CNs	$\|\sigma_h^{(2)} - \sigma_h^{(1)}\|_{0,\Omega}$	Number of vertices	Number N of CNs	$\|\sigma_h^{(3)} - \sigma_h^{(2)}\|_{0,\Omega}$
18 510	18 510	6.4×10^{-2}	5277	20 936	3.7×10^{-3}
36 424	36 424	3.9×10^{-2}	12 861	51 186	1.5×10^{-3}
103 752	103 752	1.5×10^{-2}	21 343	85 035	8.9×10^{-4}
126 669	126 669	1.2×10^{-2}	55 116	219 844	2.2×10^{-4}

$$\Omega = \left\{(x_1, x_2, x_3) \in \mathcal{R}^3 \mid (0 \le x_1 \le 1)\ \&\ (-0.5 \le x_2 \le 0.5)\ \&\ (0 \le x_3 \le 1)\right\}$$

Since the solution is basically two-dimensional, we expect the mesh to be refined as in the two-dimensional case in the x_1–x_2 plane to capture the vortices in the solution. In the perpendicular direction however, there is no variation in the solution and consequently, the elements should be stretched accordingly. This is precisely what can be observed in Fig. 8 where a cross section of the solution in the plane $x_3 = 1/2$ is presented. The three-dimensional mesh contains 17 275



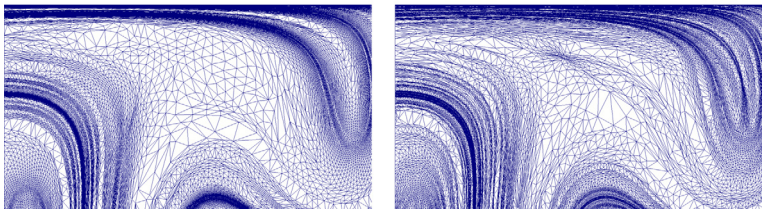


Fig. 5. Close up views of the adapted meshes with quadratic discretizations. Left: meshes adapted using the displacement field ($N = 225\,240$ CNs). Right: meshes adapted using the FTLE field ($N = 219\,844$ CNs).

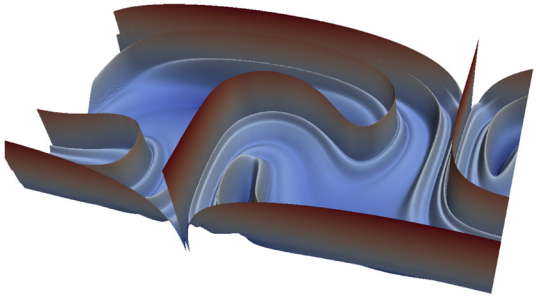


Fig. 6. FTLE field: quadratic discretization with $N = 219\,844$ CNs.

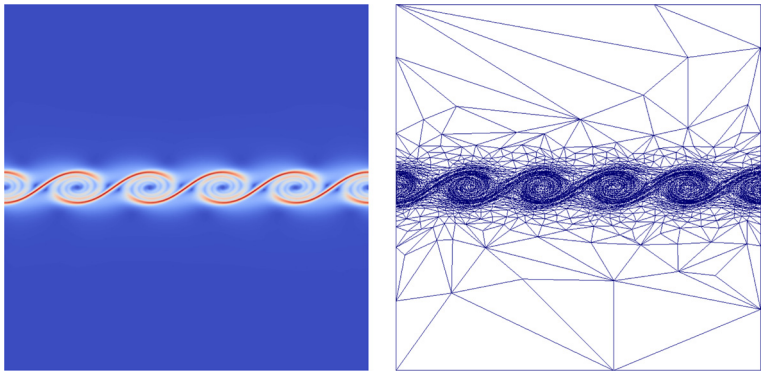


Fig. 7. Cat's eyes problem 2D: FTLE field and adapted mesh.

vertices and 137 584 CNs and was also obtained after only four iterations of FTLE computations and remeshings. Note that on the cross section, the illustrated "mesh" is in fact the intersection of the three-dimensional mesh with the plane of the cross section. This is why the mesh does not seem to follow the FTLE field as well as in the two-dimensional case. The computed solution is however very similar to the one obtained in 2D.

5. Conclusions

We have presented an adaptive method which requires only the first order derivatives of the displacement field. It results in a significantly reduced system of ordinary differential equations for the computation of the FTLE field while preserving its accuracy.

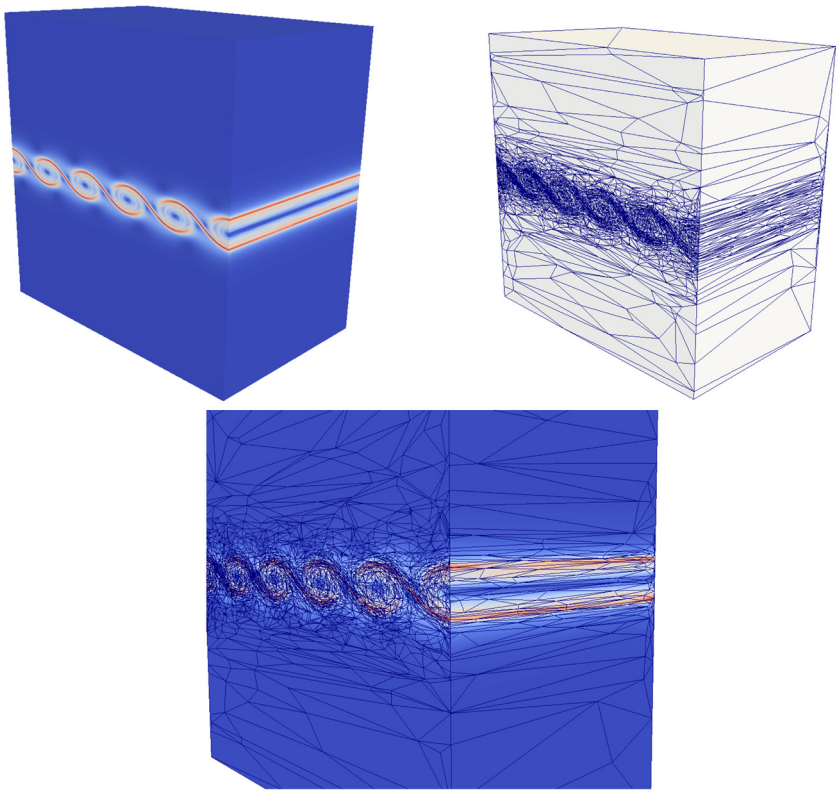


Fig. 8. Cat's eyes problem 3D: FTLE field, cross section of the mesh and close view.

The identification of flow structures by the finite-time Lyapunov exponent (FTLE) has been revisited, to enhance accuracy and to reduce the computational cost. The methodology, based on a set of 6 differential equations in 2D and 12 differential equations in 3D, yields the flow map and its first order derivatives, and intrinsically provides an interpolation error as a quality indicator of the displacement field. This results in improved nodal accuracy of the components of the Cauchy–Green tensor and its eigenvalues. Specifically, the methodology has been applied to linear and quadratic approximations.

This approach is metric-free and can be theoretically used in conjunction with P^k approximation of the displacement field. This is certainly a path to follow to reduce the number of nodes and the anisotropic ratio near *LCS*. This would certainly benefit mesh adaptation software robustness and efficiency. In theory, it should have no impact on the computation of the flow map and its gradient, and should improve FTLE field quality.

The efficiency of the method was illustrated with only two examples. Results obtained from analytical solutions, some of which have already been used in previous works, reveal that the current framework of anisotropic mesh adaptation is well suited to the computation of FTLE fields.

We have not reported on the computational times since, at the present exploratory stage of our research, the different components of the computations are loosely coupled (file based exchange of information data). The system of ODEs is solved in Matlab while the adaptation step is done in C++ in our finite element code. We would like to emphasize that the adaptation procedure is fully parallel and consequently, the extra computational time is reduced to its minimum. A fair comparison of the computational times would also require computing solutions on a series of uniform meshes until we reach a similar accuracy. As a first estimate, we have computed the FTLE field on a uniform mesh (quadratic discretization) with more than 261 000 CNs. The estimated error was around 3.37×10^{-2}, more than a hundred times higher than that on our finest adapted meshes (219 844 Cns) as indicated in Table 2. To obtain a similar accuracy, we estimate that a uniform mesh would require more than 6.2 million CNs. We also want to mention that we have already made this kind of comparison on a different (time-dependent) problem in [3]. We have established that for **a given level of accuracy**, the total computational time is reduced even when taking into account the computational time associated to mesh adaptation. We strongly believe that similar conclusions will be obtained when a strongly coupled and minimally optimized code will be available.

Acknowledgements

The authors wish to acknowledge the financial support of the Natural Sciences and Engineering Research Council of Canada (NSERC).

References

[1] F. Alauzet, P. Frey, Estimateur d'erreur géométrique et métriques anisotropes pour l'adaptation de maillage. Partie I: aspects théoriques, Technical Report 4759, INRIA, 2003.
[2] R.E. Bank, K.R. Smith, A posteriori error estimates based on hierarchical bases, SIAM J. Numer. Anal. 30 (4) (1993) 921–935.
[3] Y. Belhamadia, A. Fortin, Y. Bourgault, On the performance of anisotropic mesh adaptation for scroll wave turbulence dynamics in reaction–diffusion systems, J. Comput. Appl. Math. 271 (0) (2014) 233–246.
[4] R. Bois, M. Fortin, A. Fortin, A fully optimal anisotropic mesh adaptation method based on a hierarchical error estimator, Comput. Methods Appl. Mech. Eng. 209–212 (2012) 12–27.
[5] R. Bois, M. Fortin, A. Fortin, A. Couët, High order optimal anisotropic mesh adaptation using hierarchical elements, Eur. J. Comput. Mech. (Rev. Eur. Méc. Numér.) 21 (1–2) (2012) 72–91.
[6] Y. Bourgault, M. Picasso, F. Alauzet, A. Loseille, On the use of anisotropic a posteriori error estimators for the adaptive solution of 3D inviscid compressible flows, Int. J. Numer. Methods Fluids 59 (2009) 47–74.
[7] S. Bélanger, Adaptation de maillage anisotrope, Master's thesis, Département de génie mécanique, École Polytechnique de Montréal, 2010.
[8] M. Castro-Diaz, F. Hecht, B. Mohammadi, O. Pironneau, Anisotropic unstructured mesh adaptation for flow simulations, Int. J. Numer. Methods Fluids 25 (1997) 475–491.
[9] T. Coupez, Metric construction by length distribution tensor and edge based error for anisotropic adaptive meshing, J. Comput. Phys. 230 (7) (2011) 2391–2405.
[10] E.F. D'Azevedo, R.B. Simpson, On optimal triangular meshes for minimizing the gradient error, Numer. Math. 59 (1) (1991) 321–348.
[11] W.G. Habashi, J. Dompierre, Y. Bourgault, D. Ait Ali Yahia, M. Fortin, M.-G. Vallet, Anisotropic mesh adaptation: towards user-independent, mesh-independent and solver-independent CFD. Part I: general principles, Int. J. Numer. Methods Fluids 32 (2000) 725–744.
[12] G. Haller, Distinguished material surfaces and coherent structures in three-dimensional fluid flows, Phys. D, Nonlinear Phenom. 149 (4) (2001) 248–277.
[13] M. Hecht, B. Mohammadi, Mesh adaptation by metric control for multi-scale phenomena and turbulence, in: 35th Aerospace Sciences Meeting & Exhibit, 1997, Reno, USA, number 97-0859.
[14] J.-F. Lagüe, F. Hecht, Optimal mesh for P1 interpolation in H1 seminorm, in: Philippe P. Pebay (Ed.), Proceedings of the 15th Int. Meshing Roundtable, Springer, 2006, pp. 259–270.
[15] P. Miron, J. Vétel, A. Garon, M. Delfour, M. El Hassan, Anisotropic mesh adaptation on Lagrangian coherent structures, J. Comput. Phys. 231 (19) (2012) 6419–6437.
[16] C.B. Moler, Numerical Computing with Matlab, SIAM, Philadelphia, 2004.
[17] J.M. Ottino, The Kinematics of Mixing: Stretching, Chaos and Transport, Cambridge Texts in Applied Mathematics, Cambridge University Press, 1989.
[18] S. Shadden, F. Lekien, J. Marsden, Definition and properties of Lagrangian coherent structures from finite-time Lyapunov exponents in two-dimensional aperiodic flows, Phys. D, Nonlinear Phenom. 212 (3–4) (2005) 271–304.
[19] T.H. Solomon, J.P. Gollub, Passive transport in steady Rayleigh–Bénard convection, Phys. Fluids 31 (6) (1988) 1372–1379.
[20] J.T. Stuart, On finite amplitude oscillations in laminar mixing layers, J. Fluid Mech. 29 (3) (1967) 417–440.
[21] M. Yano, D.L. Darmofal, An optimization-based framework for anisotropic simplex mesh adaptation, J. Comput. Phys. 231 (22) (2012) 7626–7649.
[22] Z. Zhang, A. Naga, A new finite element gradient recovery method: superconvergence property, SIAM J. Sci. Comput. 26 (4) (2005) 1192–1213.
[23] O.C. Zienkiewicz, J.Z. Zhu, The superconvergent patch recovery and a posteriori error estimate, part I: the recovery technique, Int. J. Numer. Methods Eng. 33 (1992) 1331–1364.
[24] O.C. Zienkiewicz, J.Z. Zhu, The superconvergent patch recovery and a posteriori error estimate, part II: error estimates and adaptivity, Int. J. Numer. Methods Eng. 33 (1992) 1365–1382.